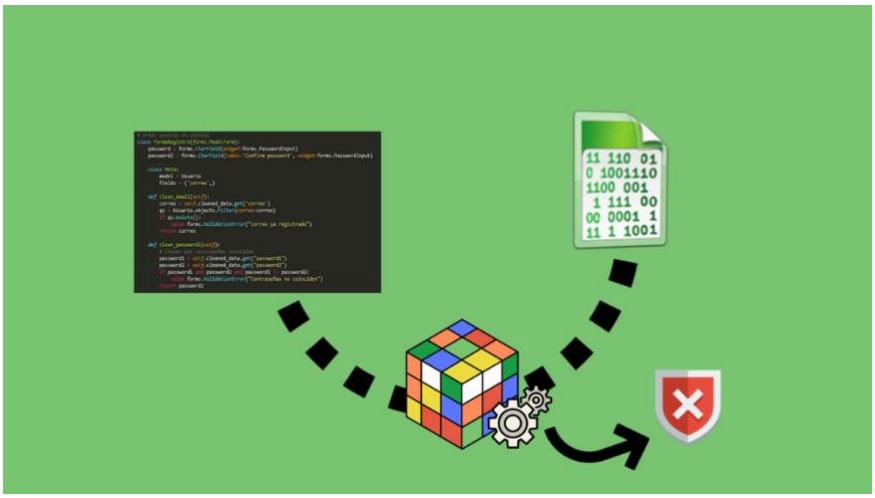
AUTÓMATAS Y COMPILADORES

UNIDAD

2



AUTÓMATAS FINITOS

© **Se prohíbe la reproducción total o parcial de esta obra, por cualquier medio, electrónico o mecánico sin la autorización de la Dirección de Educación a Distancia (UDB Virtual).**

Dirección de Educación a Distancia ©

Apartado Postal, 1874, San Salvador, El Salvador

Tel: 2251-8200 ext: 1743

Sumario

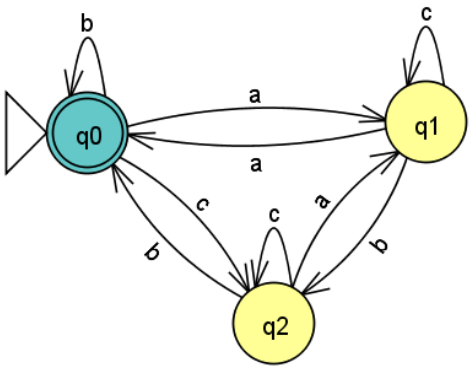
Introducción a la Unidad 2 4
Clase 5| Concepto de autómatas finitos y máquinas de estado 5
Clase 6| Autómata finito determinista 15
Clase 7| Simplificación de autómatas finitos deterministas 29
Clase 8| Autómatas finitos no determinista 38
Referencias citadas en UNIDAD 2 46
Glosario de los términos citados en la UNIDAD 2 47

Introducción a la Unidad 2

En la unidad 1, se abordaron las temáticas sobre la teoría de los conjuntos partiendo de la importancia de los conceptos y conocimientos matemáticos aplicados en los lenguajes formales, entornos gramaticales, notaciones formales, diagramas de Venn, operaciones, entre otros.

En la unidad 2, estudiaremos las máquinas abstractas más simples, los autómatas finitos, los cuales están en relación con los lenguajes regulares. Representaremos a los autómatas finitos por medio de diagramas y términos matemáticos utilizando las notaciones formales vistos en la unidad 1. Aprenderemos a interpretar, identificar y construir autómatas finitos deterministas y no deterministas. Para finalmente utilizar los métodos de simplificación de los autómatas finitos deterministas aplicando las reglas de equivalencia en autómatas.

Al finalizar esta unidad el estudiante logrará adquirir las competencias a través de ejemplos y ejercicios que desarrollaremos a lo largo de la unidad.





Clase 5| Concepto de autómatas finitos y máquinas de estado

5. Autómatas finitos y máquinas de estado

La teoría de los autómatas se ocupa de la clasificación sistemática y el estudio de varios tipos de máquinas abstractas que realizan el procesamiento de información secuencial. Del conjunto de máquinas abstractas estudiaremos los autómatas finitos que constituyen el conjunto de máquinas más simples. Los autómatas finitos constituyen un modelo útil para muchos tipos de hardware y software bajo los lenguajes regulares.

5.1 Concepto de autómatas finitos



Para Jurado (2008) “un autómata es un dispositivo que procesa cadenas de símbolos que recibe como entrada, cambiando de estado y produciendo una salida que, en algunos casos, puede estar formada por otra cadena de símbolos” (p. 58).

El término "máquina" se refiere a algo hecho de metal, a menudo grande y grueso, que realiza tareas repetitivas que requieren mucha energía, velocidad o precisión. Un modelo de automóvil es una imagen abstracta de un automóvil que ayuda a expresar su forma física. Sin embargo, existe otra forma de modelar una máquina: cómo funciona, conociendo la secuencia de operaciones que realiza.

5.2 Modelado de sistemas discretos

Antes de definir máquinas de estados finitos, primero consideramos situaciones de la vida real que se pueden modelar usando estas máquinas de estados. De esta forma pasaremos de lo más concreto a lo más abstracto, lo que nos permitirá entender visualmente el tema. La modelización de fenómenos y procesos permite:

- Validar hipótesis sobre los procesos descritos.
- Predecir el comportamiento futuro.
- Ejecute la simulación (tal vez en su computadora).
- Haz experimentos como "¿y si...?" sin afectar los procesos o fenómenos físicos.

5.2.1 Eventos discretos

Los **eventos discretos** son eventos cuyo estado solo se procesa en momentos específicos, separados por intervalos de tiempo, independientemente de lo que suceda en el sistema entre estos momentos. Como si el desarrollo del sistema se caracterizara por una serie de imágenes en lugar de un flujo continuo, cambiaría abruptamente de una fotografía a otra. A menudo se supone que la realidad es continua, por lo que los sistemas discretos son simplemente abstracciones de ciertos sistemas, y estamos interesados en resaltar los aspectos "discretos" de ellos. Por ejemplo, un motor de gasolina tiene cuatro tiempos: admisión, compresión, encendido y escape. Sin embargo, el pistón en realidad no está limitado a cuatro posiciones, sino a varias posiciones consecutivas. Por lo tanto, "cuatro veces" es una abstracción de la realidad.

5.2.2 Estados

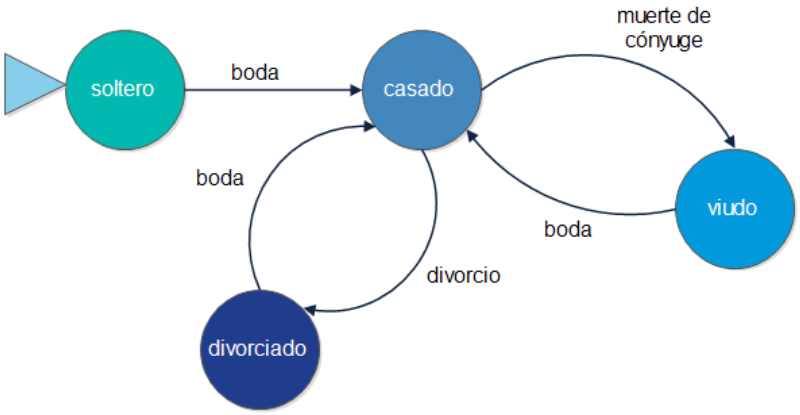
El concepto básico del modelo de eventos discretos es el ***estado***. Un estado es una condición que dura un cierto período de tiempo. Un ejemplo en la vida es el “estado civil”, una persona puede ser: soltera, casada, viuda, divorciada, etc. Cuando ocurre un hecho o una acción, puede pasar de un estado a otro, que es otro concepto básico de la discrecionalidad. Por ejemplo, cuando ocurre el evento "boda", puede cambiar el estado "soltero" a "casado".

También puede pasar de "casado" a "divorciado" a través de un acuerdo de separación. Estos modelos asumen que los estados existen durante un período de tiempo específico, pero que los eventos ocurren instantáneamente. Puede ser más o menos realista dependiendo de la situación modelada. Por ejemplo, en el caso de "divorcio", puede que no sea apropiado tratarlo como un evento inmediato ya que algunos divorcios tardan años en completarse. En este caso, el modelo se puede refinar determinando el nuevo estado "divorciado" obtenido del estado "casado" en el evento "inicio de divorcio".

La representación gráfica de modelos de estados y eventos es extremadamente práctica. Los estados se representan mediante **óvalos o círculos,** y los eventos con flechas entre los óvalos se denominan **transiciones**. Dentro de cada estado esta su nombre y al lado de las transiciones el nombre del evento asociado, como en la Figura 1. El estado en el que comienza se marca con el símbolo "▶", en este caso "soltero".

Figura 1

Modelo de estados civiles de una persona

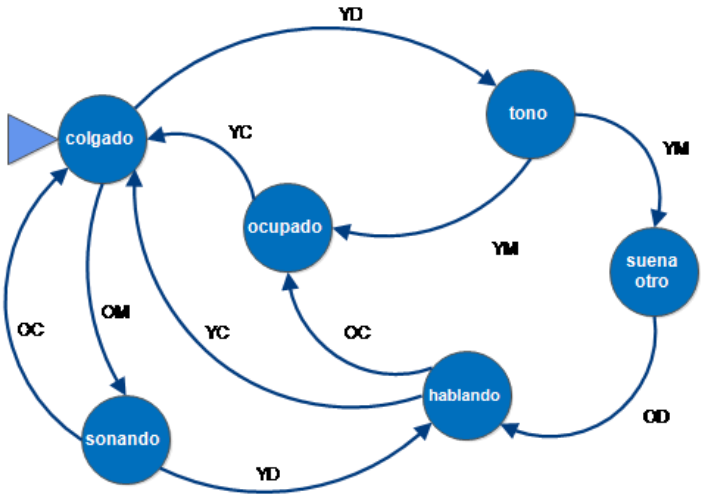


Fuente: *Nota.* Elaboración propia.

En la Figura 2 se presenta un modelo simplificado del funcionamiento de un aparato telefónico. En esta figura los nombres de los estados se refieren al dispositivo al que estoy llamando, recibiendo, etc., de lo contrario, se representa como otro ("otra persona está llamando", refiriéndose al teléfono en otra parte). En las transiciones, la "Y" inicial se refiere a acciones que hace la persona (por ejemplo, "YD" significa "Estoy contestando una llamada"), y la "O" se refiere a otro teléfono. La "C" de "YC" significa "desconectar", y la "M" significa "marcar". Por lo tanto, las transiciones YC, OC, YM, OM, YD y OD debe quedar claro.

Figura 2

Modelo de eventos discretos de un teléfono



Fuente: *Nota.* Elaboración propia.

En este ejemplo, asumimos que el estado en el que comienza el proceso es el estado inicial, por tanto, es el estado en el que el teléfono está en espera o colgado, pero aún no está conectado (sin tono). En esta situación puede ocurrir algún evento que nos lleve a un nuevo estado, por ejemplo, iniciar una llamada o descolgar el teléfono para marcar un número. Es una práctica construir modelos "correctos" de procesos reales, pero en general las siguientes recomendaciones pueden ser útiles:

a. Se hace una distinción entre eventos que se consideran instantáneos y eventos con duración crítica, estos eventos están asociados con estados. Los estados son la base para construir los modelos que hemos estudiado porque "recuerdan" las situaciones clave en las que se desarrolla el proceso.
b. Las condiciones relacionadas con los estados deben ser únicas, es decir, no se pueden probar varias condiciones al mismo tiempo. Por ejemplo, una persona que no es soltera y está casada al mismo tiempo.
c. Las condiciones relacionadas con los estados de un modelo generado deben ser comprensibles, es decir, cubren todos los casos posibles. Por ejemplo, en el modelo de estado civil, asumimos que la persona es soltera, casada o divorciada y no tiene

otra opción. Si necesitamos considerar la cohabitación como una condición más, habrá que modificar el modelo.

d. Los eventos inmediatos están asociados con eventos. En este ejemplo, levantar el teléfono (suponiendo una acción inmediata) es una transición, digamos que el usuario puede tardar un tiempo en marcar un número, por lo que hay un estado entre los dos eventos.

En el ejemplo del teléfono, asumimos que se escucha un tono de marcación inmediatamente después de descolgar el teléfono, aunque en algunas ciudades esta suposición puede ser una simplificación excesiva inaceptable. Por otro lado, en el mismo ejemplo, asumimos que la persona que contesta la llamada no lo hace inmediatamente, pero la llamada sí comienza y finaliza. Para eventos con duración, los eventos de inicio y finalización deben definirse como en el ejemplo de divorcio mencionado anteriormente. Por supuesto, decidir qué eventos son inmediatos y cuáles son tardados depende completamente de lo que es importante en el problema particular que se modela.

Los errores más comunes al modelar estados y eventos son:

- Confundir estados con eventos; por ejemplo, tener un estado “salir de casa”, que lógicamente corresponde a un evento instantáneo.
- Recomendar grupos de estados no exclusivos, es decir duplicados, como “Se encuentra en el trabajo” y “Se encuentra fuera de casa”, se pueden verificar al mismo tiempo, lo que no se puede en los estados.
- El conjunto de estados propuestos no debe de faltar ningún caso o situación bajo consideración.

En situaciones muy complejas, donde varios procesos evolucionan concurrentemente, el modelado de eventos discretos por medio de estados y eventos no es adecuado, pues los diagramas son demasiado grandes.

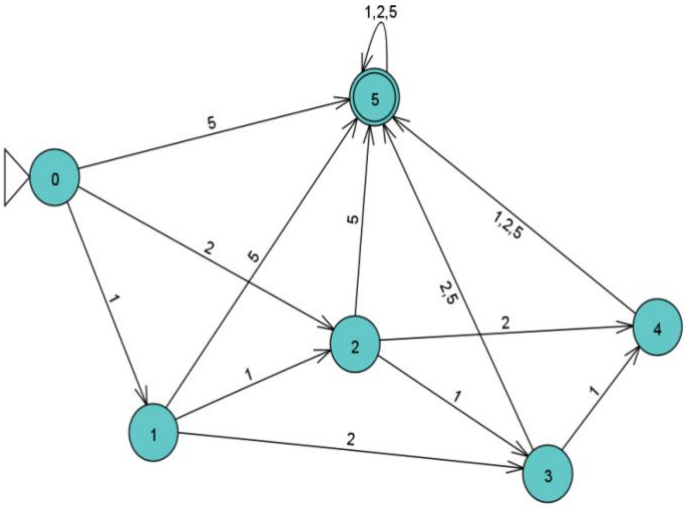
El objetivo de algunos modelos de estados y eventos es identificar secuencias de eventos "buenas" o "aceptadas" para que puedan distinguirse de las secuencias "malas" o "rechazadas". Por ejemplo, suponga que desea simular el funcionamiento de una máquina

expendedora de bebidas enlatadas. Esta máquina acepta monedas 1, 2 y 5 y el precio de cada lata es de 5. Supondremos que el evento "1" es el evento de introducir 1 moneda en la máquina, el evento "2" es la moneda 2, y así sucesivamente.

El primer problema a resolver para el diseño de nuestro modelo es cómo se ven los estados. Por supuesto, el estado inicial es 0. Con estas ideas en mente, podemos construir un gráfico de estado y evento como el de la Figura 3.

Figura 3

Modelo con estados finales



Fuente: *Nota.* Elaboración propia.

Muchas transiciones en tal patrón son obvias, como la transición del estado "1" al "3" después de la moneda de valor 2. Un aspecto muy importante del modelo de la Figura 3 es que el estado "5" es una condición especial llamada estado final y está representado por un óvalo con dos trazos. Los estados finales indican que una vez que se alcanzan, la secuencia de eventos que conducen a ellos puede considerarse "aceptable". Por ejemplo, en una máquina de bebidas en latas, la secuencia de eventos "2", "1", "2" puede considerarse aceptable porque es 5. La figura 3 muestra la secuencia indicada pasando por los estados 0, 2, 3 y 5, siendo este último el estado final. De esta forma, el esquema nos

permite distinguir cadenas válidas de cadenas rechazadas como "1", " 2", " 1", dando como resultado un estado de 4 que no es un estado final. Debemos considerar que al ingresar "5", "5" y "5 " también son aceptables, porque desde el punto de vista de la máquina se puede adquirir 3 latas.

Se pueden dar otros ejemplos de modelos:

- Podemos pensar en un semáforo como una máquina que puede moverse a través de los estados de rojo, verde y amarillo.
- Podemos pensar en la silla como una máquina que puede pasar por estados desocupada, ocupada y no disponible.
- Una alarma antirrobo de un carro, los estados pueden ser; activada, desactivada, sonando.

5.3 Máquinas de estados finitos

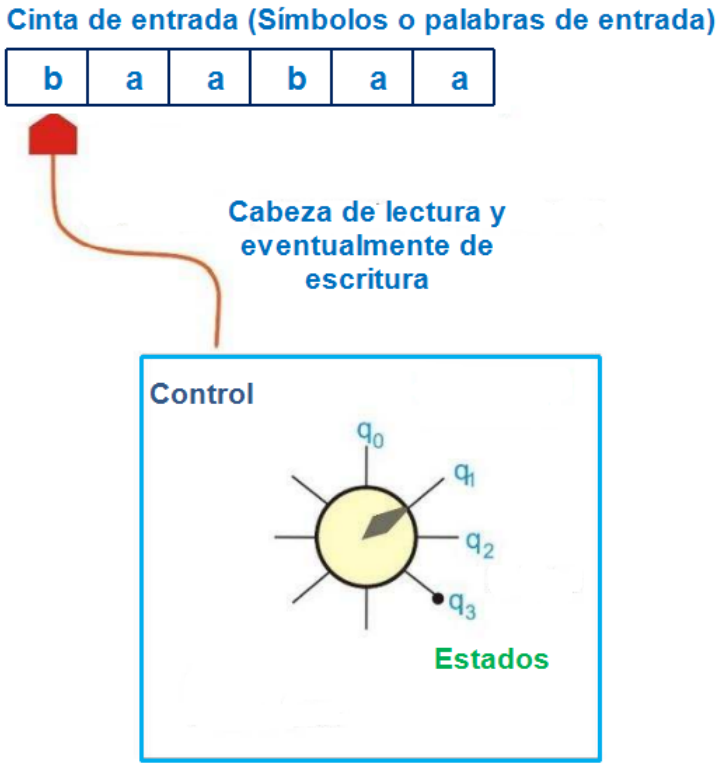
De ahora en adelante, veremos modelos de estados y eventos que son un poco más abstractos que los que hemos visto antes. Volvamos al ejemplo de la máquina tragamonedas que vimos antes. En este modelo, podemos reconocer secuencias de eventos "aceptables", como la secuencia de monedas 2, 2, 1, versus secuencias inaceptables como 1, 1, 1. A partir de ahora, los nombres de eventos estarán formados por un carácter y los llamaremos transiciones en lugar de "evento". Así que en lugar del evento "1" tendremos una transición, por ejemplo, con el símbolo "1". Por supuesto, la elección del símbolo que debe tomarse como nombre de la transición es una decisión arbitraria. Además, las secuencias de eventos se representarán mediante la concatenación de caracteres, es decir, palabras. Así, en el ejemplo de la máquina, la palabra "1121" representa la secuencia de eventos "introducir 1", " introducir 1", " introducir 2", " introducir 1". Desde el punto de vista abstracto que aplicaremos a partir de ahora, nuestra máquina puede pensarse como un dispositivo con los siguientes componentes:

- Cinta de entrada.
- Una cabeza de lectura y eventualmente de escritura.
- Control.

La Figura 4 muestra los componentes de una máquina de estados.

Figura 4

Componentes de una máquina de estados abstracta



Fuente: *Nota.* Elaboración propia.

Se coloca una cinta que contienen los caracteres que componen la entrada, y cuando la cabeza lectora se coloca sobre los caracteres los "lee" y envía esta información al controlador; también se puede mover un lugar hacia la derecha o hacia la izquierda según el tipo de máquina.

Un control mostrado como una perilla, le dice a la cabeza lectora cuándo moverse hacia la derecha. Las posiciones del control cambian según la entrada y algunas posiciones pueden ser finales, por ejemplo, marcadas con un punto (q3) se consideran especiales porque permiten especificar una palabra aceptable o rechazada, como veremos más adelante.

Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender las máquinas de estado, como lo es la máquina de Turing:

Iraic. (2 de junio de 2022). *Lenguajes y Autómatas - MT Ejercicio 1.* [Archivo de Vídeo]. Youtube. https://www.youtube.com/watch?v=aA1nfL9DqN4

5.4 Funcionamiento de los autómatas finitos

Concepto

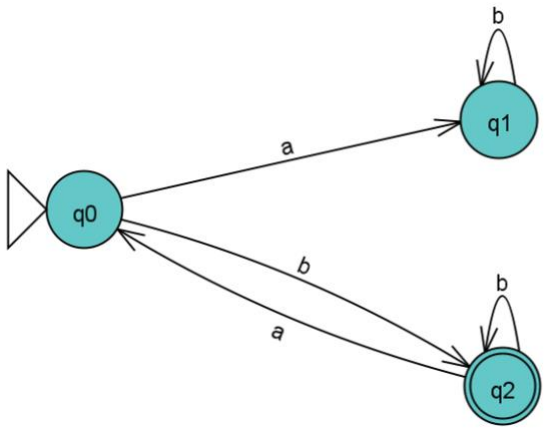
"Un autómata finito tiene un conjunto de estados y su control pasa de un estado a otro en respuesta a las entradas externas" (Hopcroft et al., 2008, p. 49).

El funcionamiento de una máquina de estados finitos es hacer la transición de un estado a otro a medida que recibe símbolos de palabras de la entrada. Este proceso se puede rastrear fácilmente en un gráfico de estado. Simplemente pasa de un estado a otro siguiendo las flechas de transición para cada carácter de la palabra de entrada, comenzando desde el estado inicial.

Por ejemplo, supongamos que tenemos un autómata de la Figura 5 e ingresa la palabra "*bb*". El autómata parte del estado q0 (que es el estado inicial), y después de recibir la primera *b*, pasa al estado q2, porque la flecha con la letra *b* va de q0 a q2 en el gráfico. Luego, al recibir la segunda *b* de la palabra de entrada, pasará del estado q2 a sí mismo, como se observa con una flecha que regresa de q2 al mismo estado.

Figura 5

Notación gráfica de un Autómata Finito



Fuente: *Nota.* Elaboración propia.

El camino tomado en el gráfico de estado se puede visualizar como un "camino o trayectoria" tomado de un estado a otro. Por ejemplo, para la máquina de estados anterior, el camino seguido por "*ab"* incluye la secuencia de estados q0, q1, q1. Los estados son el único medio disponible para que AF recuerde eventos (como los caracteres que se han leído hasta el momento); es decir, son máquinas con memoria limitada. Finalmente, las computadoras digitales son máquinas con memoria limitada, aunque el número de estados posibles de su memoria puede ser muy grande.

Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender un poco más sobre las máquinas de estado finito:

Luis Carlos Contreras. (1 de agosto de 2022). *Máquinas de estado finito (Autómatas).* [Archivo de Vídeo]. Youtube. https://www.youtube.com/watch?v=vlbZRJ96Ei8

Clase 6| Autómata finito determinista

6. Autómata finito determinista (AFD)

Concepto

Para Hopcroft et al. (2008) un autómata finito determinista, que es aquel que sólo puede estar en un único estado después de leer cualquier secuencia de entradas. El término determinista hace referencia al hecho de que para cada entrada sólo existe uno y sólo un estado al que el autómata puede hacer la transición a partir de su estado actual. (p.56)

A continuación, presentaremos un formato matemático para representar información similar a un gráfico de estados. Dado que se utiliza el término matemático en lugar de la imagen, podemos utilizar ***notación formal***. En particular, usamos los conceptos de la teoría de conjuntos vistos en la unidad 1.

Una máquina de estados finitos M es un quíntuplo $(K, \Sigma, \delta, s, F)$, donde:

- K es un conjunto de identificadores (símbolos) de estados.
- Σ es el alfabeto de entrada.
- $s \in K$ es el estado inicial.
- $F \subseteq K$ es un conjunto de estados finales o de aceptación.
- $\delta: K \times \Sigma \rightarrow K$ es la función de transición, que a partir de un estado y un símbolo del alfabeto obtiene un nuevo estado.

La función de transición indica a qué estado ir, partiendo del estado actual y el carácter que se lee. Tenga en cuenta que δ es una función, no solo una relación; significa que para un estado y carácter alfabético dado habrá uno y sólo un estado subsiguiente. La característica que nos permite saber siempre cuál será el siguiente estado se llama ***determinismo***.

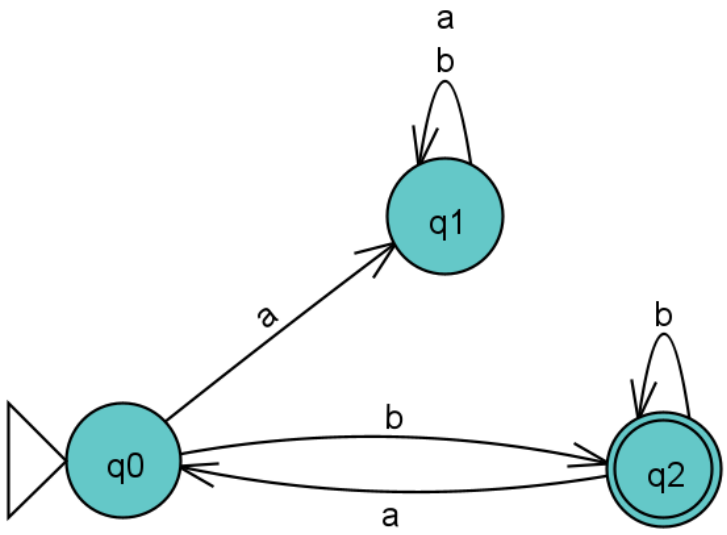
Ejemplo. El autómata finito determinista de la Figura 6, puede ser expresado formalmente como:

M = (K, Σ, δ, q0, F), donde:

- K = {q0, q1, q2}
- Σ = {a, b}
- δ = {((q0, a), q1), ((q0, b), q2), ((q1, a), q1), ((q1, b), q1), ((q2, a), q0), ((q2, b), q2)}
- F = {q2}

Figura 6

Notación gráfica de AFD



Fuente: *Nota.* Elaboración propia.

Concepto

"Una tabla de transiciones es una representación tabular convencional de una función" (Hopcroft et al., 2008, p.58).

La función de transición δ puede ser expresada mediante una tabla, en la Tabla 1 se presenta la transición δ y en la Tabla 2, se presenta la tabla de transición con respecto al alfabeto como entrada {a,b}.

Tabla 1

Tabla de transición δ

q	σ	δ (q, σ)
q_0	a	q_1
q_0	b	q_2
q_1	a	q_1
q_1	b	q_1
q_2	a	q_0
q_2	b	q_2

Fuente: *Nota.* Elaboración propia.

Tabla 2

Tabla de transición para Σ = {a, b} del AFD

	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_1	q_1
$^*q_2$	q_0	q_2

Fuente: *Nota.* Elaboración propia.

Tenga en cuenta que q0 se agrega una flecha "→" para indicar que es un estado inicial, y "*" se agrega a q2 para indicar que es un estado final.

Es fácil ver que la diferencia entre el diagrama de estado y el DFA en notación formal es solo en notación, la información es exactamente la misma, por lo que es fácil cambiar de una a otra.

Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender un poco más sobre la definición formal de los AFD's:

Juancar Molinero. (16 de febrero de 2014). *Definición Formal de un Autómata Finito Determinista (AFD).* [Archivo de Vídeo]. Youtube. https://www.youtube.com/watch?v=P0AxQvJcN2Q

6.1 Restricción de los AFD's

Tanto en el diagrama de estado como en la representación formal se deben dar condiciones para que tengamos un AFD válido; en particular, el número de transiciones provenientes de cada estado debe ser igual al número de caracteres del alfabeto, ya que δ es una función definida para todas las entradas posibles. En el ejemplo de la Figura 6 visto anteriormente, donde el alfabeto es {a, b}, deben salir exactamente dos transiciones de cada estado, una para *a* y otra para *b*, la restricción se aplica solo a aquellas que salen. Otra condición es que debe haber exactamente un *estado inicial*. En cambio, el número de estados de terminación puede ser cualquiera, incluido cero, hasta |K| (número de estado). En notación formal, también necesitamos realizar un seguimiento de las transiciones, que ahora no están representadas por flechas, sino por los elementos del conjunto δ transiciones.

El autómata de la Figura 6, al colocar la cadena de entrada *bb*, la operación comienza en el estado inicial q0; luego, después de obtener la primera palabra *b* a través de la transición ((q0, b), q2), pasa a q2, luego, después de obtener la segunda palabra de entrada *b*, a través de la transición ((q2, b), q2), pasa al estado q2 (no se mueve). Generalmente, si el AFD está en q y toma el símbolo σ, cambia a q' sí y solo si $\delta(q, \sigma) = q'$, esto es, si $((q, \sigma), q') \in \delta$.

6.2 Lenguaje aceptado por un AFD

Las máquinas de estado que hemos visto se pueden utilizar para reconocer ciertas palabras y distinguirlas de otras. Suponemos que el AFD reconoce o acepta una palabra si se cumplen las siguientes condiciones:

- Todos los caracteres de la palabra de entrada especificada se utilizan de acuerdo con la transición y, por lo tanto, pasarán de un estado a otro.
- Al final de la palabra, el estado que alcanza es uno de los estados finales del autómata (estados con círculos dobles en el diagrama o parte del conjunto F en la representación formal).

Sea un *AFD = (K, Σ, δ, q0, F)*, se expresa que una palabra x es aceptada o reconocida por el AFD si $f'(q_0, x) \in F$.

Así, en el ejemplo de la Figura 6, el autómata acepta la palabra *bb*, pues al terminar de consumirla se encuentra en el estado q_2, el cual es final.

El concepto de *lenguaje aceptado* es una simple extensión de aquel de palabra aceptada. El lenguaje aceptado por una máquina M o el lenguaje asociado a un AFD es el conjunto de palabras aceptadas por dicho autómata.

$L = \{x / x \in \Sigma^* \text{ and } f'(q0, x) \in F\}$

- Si $F = \{\} = \emptyset \Rightarrow L = \emptyset$
- Si $F = K \Rightarrow L = \Sigma^*$

Otra definición:

$$L = \{x / x \in \Sigma^* \text{ and } (q0, x) \rightarrow (q, \lambda) \text{ and } q \in F\}$$

Dónde: λ indica que la cadena de entrada ha sido leída completamente

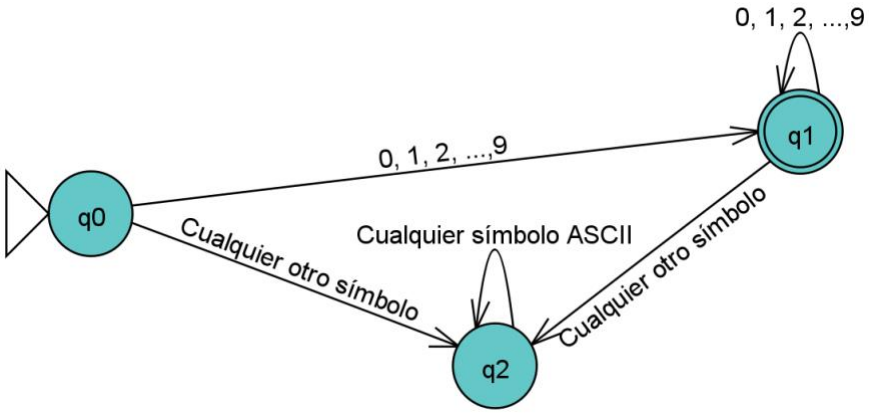
Así, el autómata de la Figura 6 acepta las palabras que empiezan con a, así como las palabras que contienen *aa*, y también las que terminan en *b*, como por ejemplo *abab*, *aaaaa*, *baaa*, etc. En cambio, no acepta baba ni *bba*, *babba*, etc. Observe que tampoco acepta la palabra vacía ε. Para que un AFD acepte ε se necesita que el estado inicial sea también final.

6.3 Aplicaciones de los AFD's

Mediante los AFD's, podemos representar las reglas para construir los tokens de un lenguaje de programación. En otras palabras, podemos especificar los aspectos léxicos del lenguaje. Por ejemplo, en la Figura 7 se muestra el diagrama de un autómata cuyo lenguaje de entrada es la tabla de caracteres ASCII, tomando los números enteros de un lenguaje de programación.

Figura 7

AFD para números enteros



Fuente: *Nota.* Elaboración propia, a partir de Hernández et al. (2010).

Note que la ruta correspondiente al entero 79245 finaliza en un estado de aceptación.

Como podrá observar los AFD's puede ser utilizados para identificar elementos de los lenguajes de programación, como: operadores relacionales, números enteros, números reales, letras, operadores aritméticos, operadores de asignación, etc.



6.4 Métodos de diseños de AFD’s

Considere el problema de generar un AFD que acepte un lenguaje. Este problema a menudo se denomina "problema de diseño". El ensayo y error es un inconveniente porque hay muchas posibilidades a considerar y es fácil cometer errores. Además, hay dos formas de cometer errores al diseñarlos:

1. "Las palabras son redundantes" significa que el autómata acepta palabras que no debería. En este caso, se dice que la solución es *incorrecta*.
2. "Palabras faltantes", es decir, hay palabras en el lenguaje que no son aceptadas por el AFD cuando estas deberían ser aceptadas. En este caso, se dice que la solución es *incompleta*.

6.4.1 Diseño por asociación de condición a cada estado

El factor más importante en el diseño de los autómatas a partir de un lenguaje es definir claramente cada estado del AFD. Debemos saber que este es un principio de diseño muy importante. El único tipo de memoria que tienen los AFD, es el estado en que se encuentra en un momento dado. Por lo tanto, el diseño de un DFA comienza con la propuesta de un conjunto de estados que "recuerdan" las condiciones importantes en el problema bajo consideración. Entonces las transiciones propuestas permitirán transiciones de un estado a otro; Esta última parte es relativamente fácil una vez que tienes los estados y sus condiciones asociadas.

Desarrollaremos un AFD que acepte palabras del alfabeto {a, b} donde el número de letras *a’s* es impar. Las condiciones pertinentes para este diseño son:

- Hasta el momento se ha obtenido un número par de *a’s* (estado P)
- El número recibido es impar (estado I)

Al comienzo del funcionamiento del AFD, no se ha recibido *a*, por lo que debemos estar en el estado P (el cero es par), lo que significa que el estado de P es inicial. Para determinar qué estado es final, debemos verificar qué estado coincide con la declaración original de las palabras recibidas. En este caso vemos que el estado I coincide (impar), por lo que es final, mientras que P no coincide (par) y no es final. Los estados P e I se muestran en la Figura 8. Este es el primer paso en el diseño de AFD.

Figura 8

Diseño de estados de un AFD



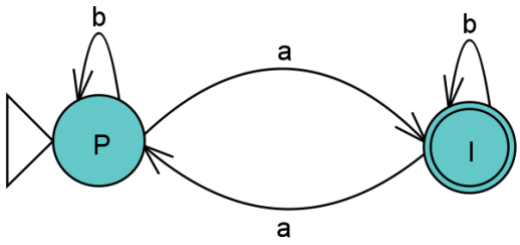
Fuente: *Nota.* Elaboración propia.

En nuestro enfoque de diseño, es importante seguir las transiciones solo después de que se hayan determinado los estados y sus propiedades.

Ahora podemos realizar un seguimiento de las transiciones, que es una tarea relativamente fácil si ya tenemos listo un borrador del estado. Por ejemplo, si estamos en *P* y obtenemos a, entonces por supuesto tenemos que ir a *I* porque el número va de par a impar. Las otras transiciones se realizan de manera similar. Los resultados se muestran en la Figura 9.

Figura 9

AFD completo



Fuente: *Nota.* Elaboración propia.

Note que las palabras *abaab*, *bbababa*, *aabbba* son aceptadas (***a's*** impares) y las palabras *aab*, *bbabba*, *abababa* son rechazadas por tener ***a's*** pares.

6.4.2 Diseño por conjuntos de estados

Podemos llevar el método de asociar una condición a cada estado un paso más allá: asociaremos las condiciones con grupos de estados, no con estados individuales. Por tanto, aumentaremos el nivel de abstracción en la etapa inicial de diseño, lo que nos permitirá resolver problemas más complejos con menor probabilidad de error. Este método implica definir primero las condiciones asociadas con la fórmula del problema, incluso si no son lo suficientemente detalladas para relacionarlas con los estados individuales.

Describimos este método aplicándolo a un ejemplo concreto: diseñe un AFD que acepte palabras del idioma en {0,1}, donde las palabras no contengan la subcadena 11, pero contengan 00.

Directamente del comunicado, indicamos las siguientes situaciones:

- Las letras utilizadas hasta ahora no contienen 00 y 11.
- Contiene 00, pero no 11
- Contiene 11

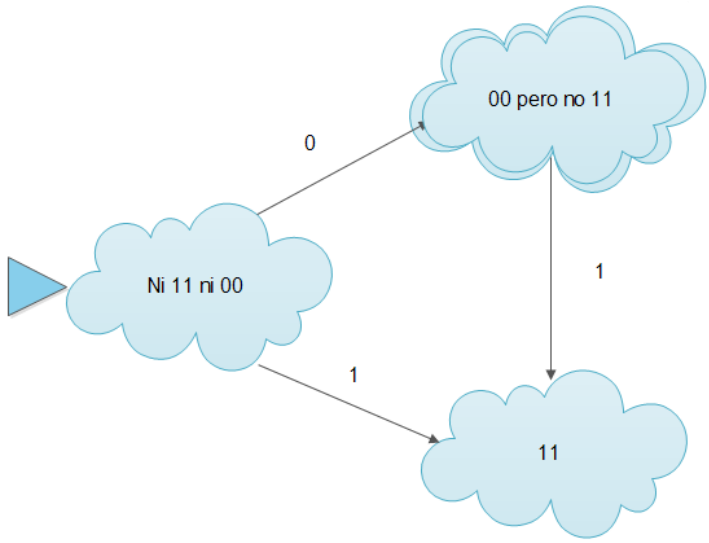
Estas condiciones satisfacen dos requisitos que deben cumplirse siempre en este tipo de construcciones:

- Las condiciones deben ser ***exclusivas***, es decir, no se pueden cumplir dos o más condiciones al mismo tiempo.
- Las condiciones deben ser ***comprensivas***, lo que quiere decir que no faltan casos por considerar.

Los grupos de estados, así como las transiciones que comprometen que se pase de un estado a otro, se representan como "nubes", en la Figura 10 se muestra el conjunto de nubes. En dicha figura también se ilustran unas nubes "dobles" para mostrar que son restricciones finales (en este ejemplo, la restricción "Contiene 00 pero no 11"), así como la restricción de inicio con un símbolo "►".

Figura 10

Grupos de estados



Fuente: *Nota.* Elaboración propia.

Este diagrama aún no es un AFD, pero están cerca. Queda por afinar cada grupo de estados teniendo en cuenta lo que sucede cuando se puede recibir cada carácter de entrada. El método para dividir cada grupo de estados "nube" en estados individuales se describe a continuación:

Las letras utilizadas hasta ahora no contienen 00 y 11.

- Inicialmente, no se reciben caracteres.
- Se recibe 0 como entrada.
- Se recibe 1 como entrada.

Contiene 00 pero no 11.

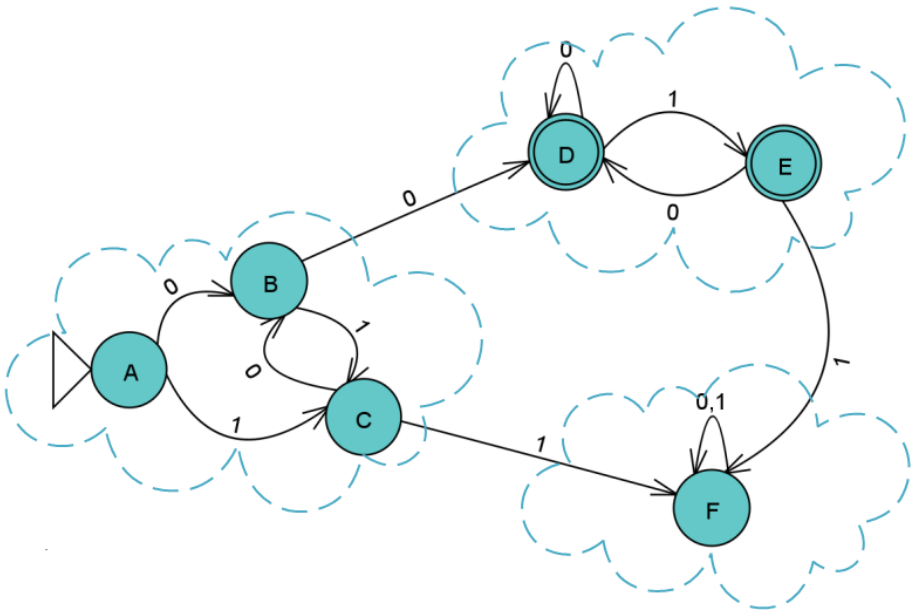
- Se recibe 0.
- Se recibe 1.

Contiene 11 (sin subcondiciones).

Esto nos da un total de 6 estados, cada uno asociado con un estado muy específico (estos son los estados "A" hasta "F"). El siguiente paso es un diseño de transición detallado que sabemos por experiencia que es relativamente simple. Los resultados se muestran en la Figura 11. Este gráfico muestra que los estados de nubes "objetivo" también son finales; esto siempre debe ser revisado.

Figura 11

Detalle de los estados



Fuente: *Nota.* Elaboración propia.

Tenga en cuenta que en este ejemplo en particular puede ser un poco difícil encontrar condiciones que estén directamente relacionadas con los estados; Por ejemplo, encontrar la condición "Las letras utilizadas hasta ahora no contienen 00 ni 11, y el resultado es 0" (estado "B" en la Figura 11) sin duda requerirá mucha creatividad. En este sentido, el diseño de grupos de estados primero permite manejar la complejidad del problema de una manera más modular e incremental.

En cualquier caso, ya sea que las condiciones se encuentren directamente para cada estado o para grupos de estados, es importante identificar primero los estados con sus

condiciones asociadas y solo luego rastrear las transiciones. La gestión del estado y la transición al mismo tiempo, muy a menudo conducen a errores.

Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender de los AFD's por medio de aplicación de la tabla de transiciones:

Jose Veliz. (18 de septiembre de 2020). *17-Ejemplo de autómatas finitos aplicando la tabla de transiciones.* [Archivo de Vídeo]. Youtube. https://www.youtube.com/watch?v=noCAGct0OsI

6.5 Autómatas equivalentes

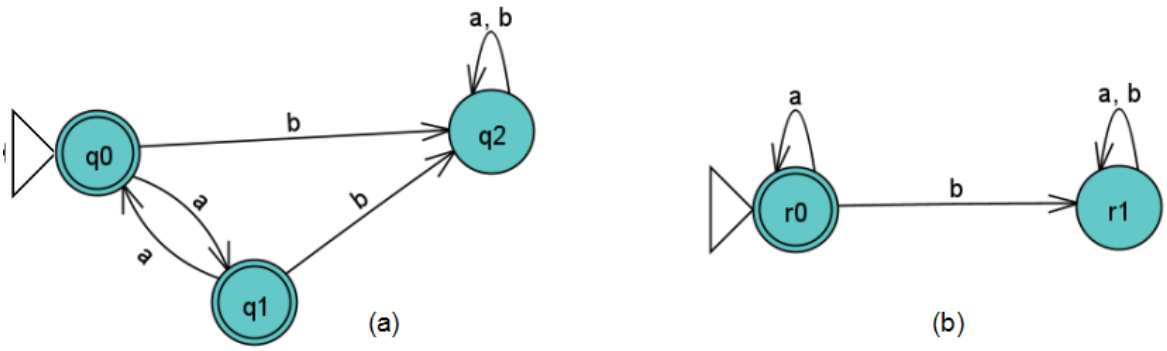
Concepto

Dos autómatas M1 y M2 son equivalentes, M1 ≈ M2, cuando aceptan exactamente el mismo lenguaje (Hernández et al., 2010).

Pero ¿es realmente posible tener múltiples AF diferentes que acepten el mismo lenguaje? La respuesta es sí. Por ejemplo, los autómatas (a) y (b) en la Figura 12, usan el lenguaje $A*$.

Figura 12

Autómatas equivalentes



Fuente: *Nota.* Elaboración propia.

Dada esta situación, dos AFD diferentes, es posible que sean equivalentes. Según la definición presentada, probar la equivalencia de dos AFD se convierte en prueba de la igualdad de los lenguajes que aceptan. Sin embargo, probar que dos lenguajes son iguales puede ser difícil con una multitud de lenguajes. Por lo tanto, se prefieren otros métodos para probar la equivalencia de los AFD. El método que se propone para los AFD se basa en el teorema de Moore.

El teorema de Moore implica la construcción de un árbol de *comparación de autómatas*. Este árbol permite transformar el problema generalmente aceptado de comparar lenguajes en un problema de comparar los estados de autómatas.

En el análisis de estados, decimos que dos estados q y q' son compatibles si ambos son estados finales o ambos estados no lo son. De lo contrario, son *estados incompatibles*.

La idea detrás del algoritmo de comparación AFD1 y AFD2 es verificar si hay alguna cadena "w" seguida de ella simultáneamente en AFD1 y AFD2 que resulte en un estado incompatible. Si tal secuencia no existe, los autómatas son equivalentes. El único problema con esta idea es que debe asegurarse de que se cubran todas las cadenas posibles de "w", generalmente infinitas. Por esta razón, se tiene que tomar en cuenta todas las combinaciones posibles de estados con un árbol de comparación. Este árbol de

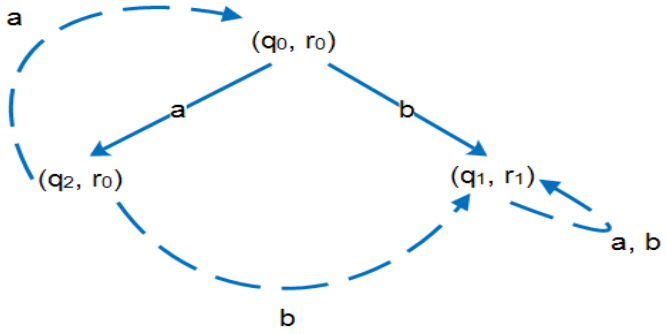
comparación se construye de la siguiente manera para dos autómatas *M = (K, Σ, δ, s, F)* y *M '= (K', Σ ', δ', s', F'):*

- Inicialmente la raíz del árbol es un par ordenado (s, s') que contiene los estados iniciales de M y M' respectivamente.
- Si en el árbol hay un par (r, r'), para cada signo de Σ se agrega el par (rσ, r'σ) como su hijo donde rσ = δ (r, σ), r'σ = δ (r', σ), si aún no existen.
- Si en el árbol aparece un par (r, r') de estados incompatibles, se rompe su estructura, diciendo que los dos autómatas no son equivalentes.
- Si no aparecen nuevos pares (rσ, r'σ) que no estén ya en el árbol, el proceso termina diciendo que los dos autómatas son equivalentes.

Ejemplo. - Sean los autómatas M y M' a partir de la Figura 12 (a) y (b) respectivamente. El árbol de comparación se muestra en la Figura 13.

Figura 13

Árbol de comparación de autómatas finitos



Fuente: *Nota.* Elaboración propia.

En dicho árbol se muestran adicionalmente, con línea punteada, las ramas que van a nodos ya existentes, como la que va de (q_2, r_0) a (q_0, r_0). Estas ramas con línea punteada no son, estrictamente hablando, parte del árbol, pero pensamos que mejoran la comprensión del diagrama. Se concluye que M y M' son equivalentes.

Clase 7| Simplificación de autómatas finitos deterministas

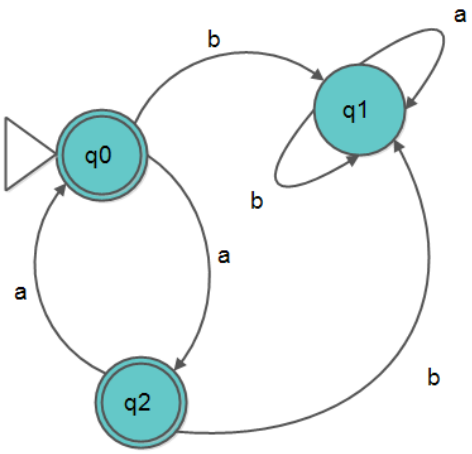
7. Simplificación de los AFD's

Una de las mejores características del AFD es que existen formas mecánicas de simplificarlo, hasta el AFD más simple y sencillo para su lenguaje. En el caso de AFD, por simplificación entendemos reducir el número de estados, pero usando el mismo lenguaje que antes de la simplificación. En adelante, la minimización significará obtener un autómata con el menor número posible de estados.

Como primer ejemplo, considere el AFD en la Figura 14. En el AFD, los estados q0 y q2 son un poco redundantes porque al recibir *a's,* el AFD continúa cambiando a q0 o q2 y al recibir *b*'s cambia a q1. Entonces puedes pensar en deshacerte de uno de ellos, por ejemplo, q2 y obtener el AFD en la Figura 15, donde hay un estado menos.

Figura 14

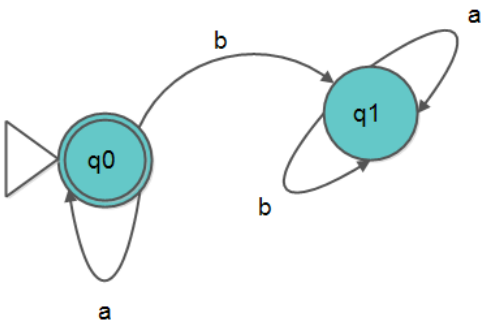
FDA para simplificar



Fuente: *Nota.* Elaboración propia

Figura 15

FDA para simplificado



Fuente: *Nota.* Elaboración propia

7.1 Estados equivalentes

Concepto

Dos estados son equivalentes, q1 ≈ q2, sí y sólo si al intercambiar uno por otro en cualquier configuración no altera la aceptación o rechazo de toda palabra. (Jurado, 2010).

Cabe destacar que, con esta definición, para comprobar la equivalencia de dos estados *p* y *q* dados, se debe comprobar cada posible palabra de entrada si al sustituirlas en las configuraciones se cambia la aceptación o no de esta palabra. Por supuesto, esto no es posible para un lenguaje infinito. La definición nos dice cuál es el estado equivalente, pero no nos dice cómo determinar si dos estados son equivalentes. Este aspecto se resuelve mediante el siguiente lema.

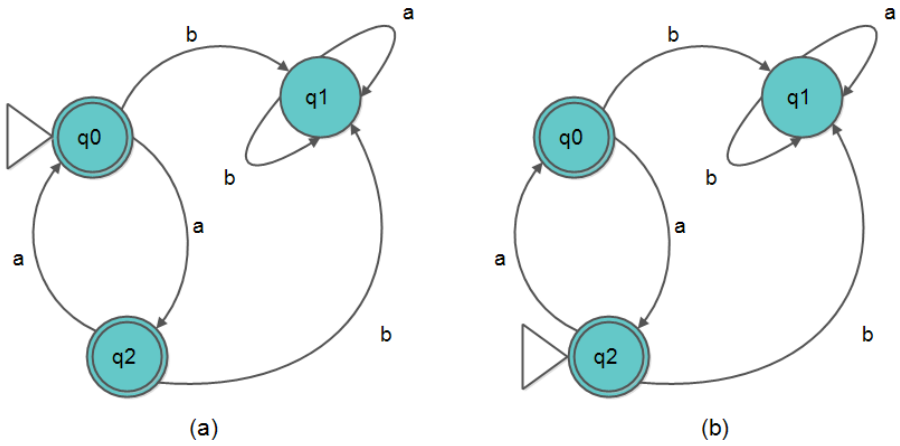
Lema: Dado un AFD $M = (K, \Sigma, \delta, q, F)$ y dos estados $q_1, q_2 \in K$, tendremos que $q_1 \approx q_2$ si $(K, \Sigma, \delta, q_1, F) \approx (K, \Sigma, \delta, q_2, F)$.

En otras palabras, para encontrar si dos estados q1 y q2 son equivalentes, se supone que ambos son los estados iniciales de dos autómatas M1 y M2 y estos autómatas se comparan entre sí. Si este último es equivalente, significa que los estados q1 y q2 son *equivalentes*.

Por ejemplo, para la máquina de la Figura16(a) para verificar si q0 ≈ q2, es necesario comparar el AFD dado con los autómatas de la Figura 16(b), donde el estado original ha sido reemplazado por otro estado a comparar. En este ejemplo, la comparación de los dos autómatas arroja resultados equivalentes, por lo que se puede concluir que los estados son redundantes.

Figura 16

Cambio de estado inicial para determinar estados redundantes



Fuente: *Nota.* Elaboración propia.

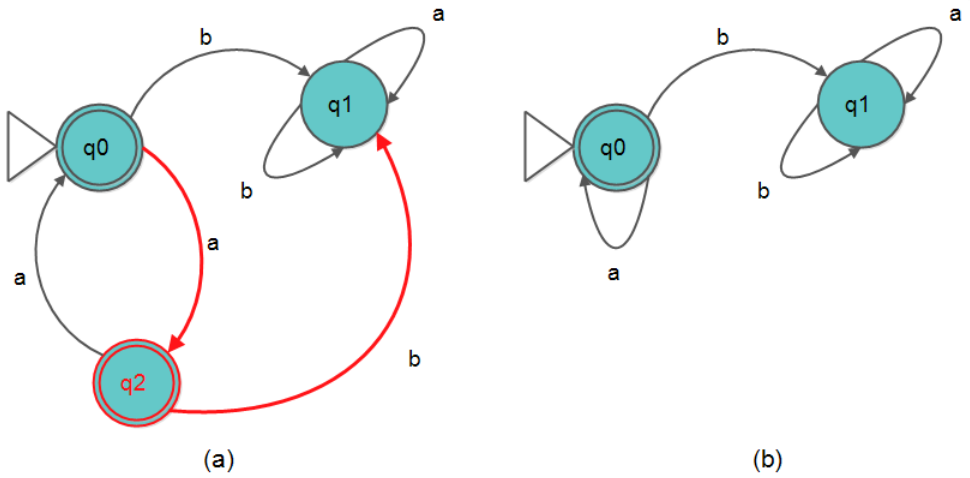
Una vez que sepa que los dos estados son equivalentes, puede considerar eliminar uno de ellos para evitar la redundancia y mejorar el rendimiento del AFD. Sin embargo, al eliminar un estado en el AFD, existe un problema con lo que se debe hacer con las flechas que conectan el estado eliminado con el resto de la máquina. Este problema se resuelve con los siguientes criterios:

- Las flechas que dejen el estado eliminado se eliminarán.
- Las flechas que alcanzan el estado de exclusión se redirigen a su estado equivalente.

Por ejemplo, en la máquina de la Figura 17(a), si probamos que q0 y q2 son equivalentes y eliminamos q2, necesitamos redirigir la flecha de q0 a q2 para que conduzca a q0 mismo (esto se convierte en un bucle). Así llegamos al autómata de la Figura 17(b).

Figura 17

Eliminación de estados equivalentes en la simplificación de un autómata



Fuente: *Nota.* Elaboración propia.

La eliminación de estados redundantes del AFD es una forma de simplificar el AFD y se puede usar repetidamente para minimizar la simplificación. Sin embargo, esto requiere mucho trabajo y, para los AFD grandes, no es práctico comprobar cada par de estados. Por lo tanto, tenemos la intención de explorar métodos más estructurados para detectar estados de redundancia y minimizar los AFD.

7.2 Simplificación por clases de equivalencias

Este método de simplificación del estado se usa con mayor frecuencia, ya que permite una organización más sistemática del trabajo. Este algoritmo, que llamaremos simplificación equivalente, procede en el orden inverso a la eliminación gradual de los estados redundantes que vimos anteriormente: en lugar de reducir el número de estados, comienza con grupos de estados o clases, es decir, se subdividen en clases hasta que la división pueda continuar.

La idea es crear clases de estados del autómata, hasta donde se sabe en ese momento, pudieran ser equivalentes entre sí. Sin embargo, al considerar las transiciones de diferentes estados de la misma clase, a veces se puede concluir que al final no deberían estar en la misma clase. En este punto, la clase en cuestión se ha dividido. Luego se verifica las transiciones de las clases resultantes para ver si es necesario subdividirlas, y así sucesivamente, hasta que se encuentre evidencia de que es necesario separar las clases.

Al final del proceso de la separación por clases, cada clase representa el estado de un autómata simplificado. Las transiciones del autómata simplificado son realizadas por las transiciones de estado contenidas en cada clase.

Formalmente, el procedimiento para AFD (K, Σ, δ, s, F) es el siguiente:

- Inicialmente, tenemos las clases F y K – F.
- Repita para cada capa.
- Sea que el estado de la clase.

Para cada estado restante *q'*, compruebe que δ(q, σ) para la misma clase es equivalente a δ(q', σ) para cada signo de σ.

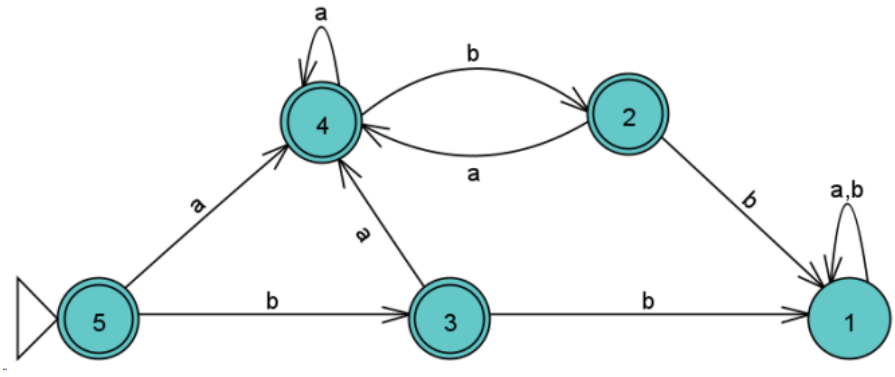
Si la respuesta es afirmativa, no es necesario dividir la clase.

Si la respuesta es "no", dividir la clase en dos subclases: la que agrupa a los estados que tuvieron transiciones “similares” a *q* y la de los estados con transiciones “diferentes” a *q* (no para la misma clase de equivalencia con el mismo símbolo σ).

Por ejemplo, considere el AFD en la Figura 18. Las dos primeras clases de equivalencia generadas contienen estados final y no final, respectivamente, que obviamente no pueden ser equivalentes (es decir, están en la misma clase de equivalencia). Estas dos capas se muestran en la Figura 19.

Figura 18

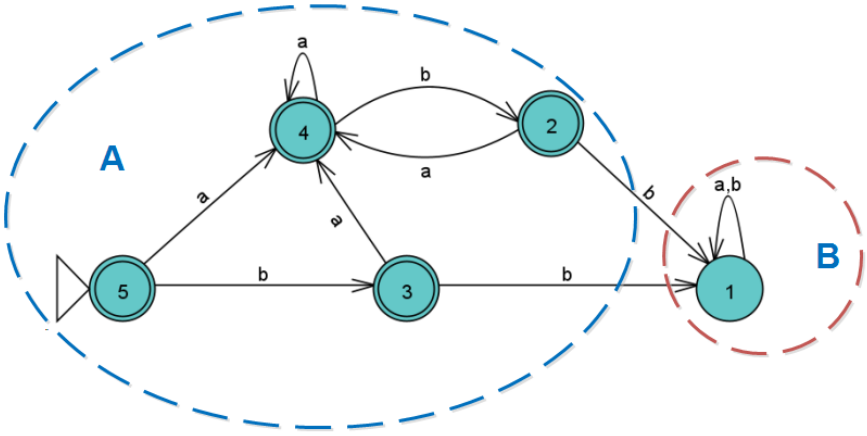
AFD a simplificar



Fuente: *Nota.* Elaboración propia.

Figura 19

Clases iniciales



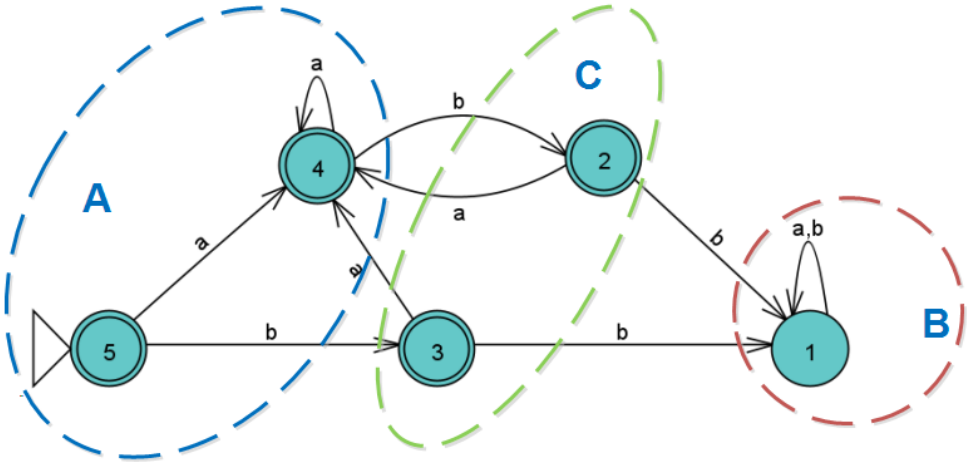
Fuente: *Nota.* Elaboración propia.

Ahora comprobaremos si todos los estados de cada capa tienen una transición "similar", lo que significa que irán a la misma clase de equivalencia. Como ejemplo, tomemos los estados 3 y 4 en la Figura 19. Después de recibir el símbolo *a*, pasamos de la capa 3 a la clase {2, 3, 4, 5}, que también se deriva del estado 4. Mientras que 3 y 4 funcionan igual. Ahora considere las transiciones con *b*: de 3 vamos a la clase {1} y de 4 pasamos a la clase

{2, 3, 4, 5}. Concluimos que 3 y 4 no pueden coexistir en la misma clase de equivalencia, por lo que la clase {2, 3, 4, 5} debe estar separada. Al realizar el mismo análisis para otros estados, dividimos {2, 3, 4, 5} en {2, 3} y {4, 5}, como se muestra en la Figura 20. En este punto ya no es posible separar ninguna de las 3 capas existentes porque sus transiciones de estado son "similares". Concluimos que estas son las mejores clases de equivalencia que se pueden generar.

Figura 20

Clases finales

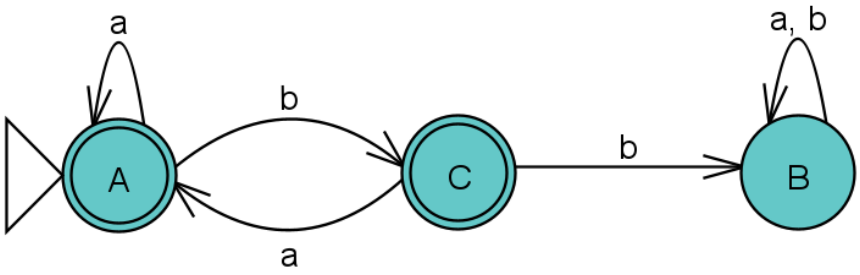


Fuente: *Nota.* Elaboración propia.

Tomando las clases de equivalencia para el AFD de la Figura 20 como estados, construimos el AFD que se muestra en la Figura 21. Tenga en cuenta que las transiciones son cualquier estado presente en cada clase; simplemente escribimos la clase de equivalencia lograda para cada carácter de entrada. El estado de inicio es equivalente a la clase de equivalencia que contiene el estado base antiguo, y los estados finales del nuevo AFD se derivan de las clases de equivalencia que contienen los estados finales del antiguo AFD.

Figura 21

AFD simplificado

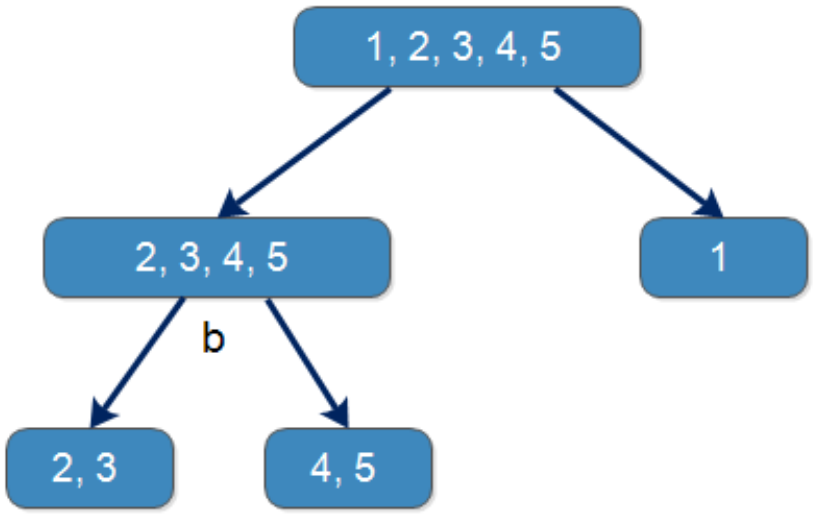


Fuente: *Nota.* Elaboración propia.

En la práctica, en lugar de dibujar líneas en el gráfico de estado, es conveniente organizar la información sobre las clases de equivalencia en forma de árbol, donde cada nodo contiene los estados de la clase de equivalencia. Inicialmente, todos los estados AFD están en la misma clase que en la raíz del árbol en la Figura 22 para el AFD de la Figura 18, y se dividen inmediatamente en final y no final.

Figura 22

Clases de equivalencia organizadas en un árbol



Fuente: *Nota.* Elaboración propia.

Luego, para el nodo {2, 3, 4, 5}, verificamos si las transiciones con símbolos de entrada, en este caso *a* y *b*, dan como resultado las mismas clases y verificamos si en el caso *b*, los estados 2 y 3 van a final, también 4 y 5 van al final por lo que el nodo se parte en dos, como se ve en el tercer nivel del dibujo. También puede ver el símbolo *b* debajo del nodo {2, 3, 4, 5}, que indica qué símbolo hace que la clase de equivalencia se divida. Con transiciones equivalentes todavía en las hojas del árbol, no hay razón para dividirlas más. Finalmente, las clases de equivalencia resultantes son {1}, {2, 3} y {4, 5} correspondientes a los tres estados que tendría el AFD reducido.



Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender la simplificación de autómatas finitos deterministas:

Juancar Molinero. (13 de abril de 2012). *Reducción de un AFD a su forma mínima.* [Archivo de Vídeo]. Youtube. https://www.youtube.com/watch?v=qgeARf15F6k&t=2s



Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender la simplificación de autómatas finitos deterministas por clases de estados:

Jose Veliz. (9 de octubre de 2020). *24-Ejemplo 1 de minimización de AFD.* [Archivo de Vídeo]. Youtube. https://www.youtube.com/watch?v=IRKY8dZEUUM

Clase 8| Autómatas finitos no determinista

8. Autómatas finitos no determinista (AFN)

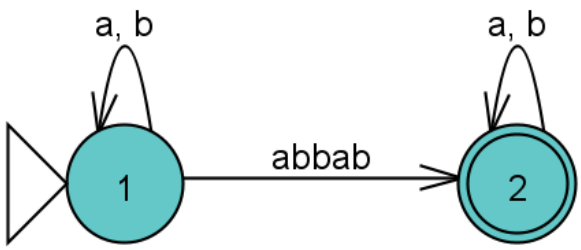
Concepto

> "Un autómata finito *no determinista* (AFN) tiene la capacidad de estar en varios estados a la vez. Esta capacidad a menudo se expresa como la posibilidad de que el autómata conjeture algo acerca de su entrada" (Hopcroft et al., 2008, p.46).

Una extensión del autómata finito es permitir que ocurra un número de flecha mayor o menor que |Σ| de cada nodo del gráfico de estado. Por lo tanto, se puede suponer que ninguna flecha corresponde a uno de los caracteres alfabéticos o que aparecen varias flechas del mismo nodo con la misma etiqueta. Las transiciones pueden incluso contener palabras de varias letras o incluso detener las palabras como un marcador. Estas máquinas de estados finitos se denominan *no determinista* o indefinidas (AFN para abreviar). Al eliminar algunas de las limitaciones de los AFD, puede ser más sencillo diseñarlas para un idioma determinado. Por ejemplo, en la Figura 23 se muestra un AFN que acepta palabras en {a, b} que contienen la subcadena "abbab".

Figura 23

AFN para palabras que contienen la subcadena "abbab"



Fuente: *Nota.* Elaboración propia.

Aquí observamos que, dado que el AFN tiene menos restricciones que un AFD, resulta que el AFD es un caso especial de la AFN, por lo que cada AFD es en realidad un AFN.

Hasta ahora, solo hemos visto las ventajas de AFN sobre AFD. Sin embargo, en los autómatas no determinísticos, es difícil saber qué camino tomar desde un determinado estado cuando se presenta el símbolo, ya que puede haber varias opciones.

Por ejemplo, tomando el autómata de la Figura 23, si se nos presenta una palabra como *abbaba*, no sabremos si debemos pasar del estado 1 al estado 2, pasando la cadena *abbab* y pasando la última *a* en 2, o gastando todas las letras de la palabra de entrada, una a una en el estado 1. El problema en este ejemplo es particularmente grave porque en un caso se alcanzó el estado final y en el otro no. Más adelante veremos cómo hacer frente a estas situaciones. Además, puede suceder que estando en el estado n y con el símbolo de entrada *a*, ninguna de las flechas marcadas con *a* salga de n (en el ejemplo de la Figura 23, esto no sucede). Estas diferencias con AFD deben reflejarse en la definición formal del AFN a continuación.

Un AFN quíntuple *(K, Σ, ∆, s, F)*, donde *K, Σ, s* y *F* tienen el mismo significado que para los AFD, y ∆, se denomina relación de transición, que es un subconjunto finito de $K \times * \times K$.

Por ejemplo, el AFN en la Figura 23 se representará matemáticamente por los siguientes quíntuplos:

({1, 2}, {a, b}, {(1, a, 1), (1, b, 1), (1, abbab, 2), (2, a, 2), (2, b, 2)}, 1, {2})

La conclusión es que ∆ es una relación, no una función. Tenga en cuenta también que el segundo elemento de la relación de transmisión es una palabra, no un carácter alfabético. Esto significa que cada triple *(q1, w, q2)* $\in$ ∆, representado en el gráfico de estado por una flecha marcada con *w*, y permite ir de q1 a q2, "usando" la subcadena w en la entrada.

Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender la definición de los autómatas finitos no deterministas (AFN):

Un Profe de Informática. (23 de abril de 2020). *Lenguajes y Autómatas - Módulo 1.5 (Autómatas finitos no-deterministas).* [Archivo de Vídeo]. Youtube. https://www.youtube.com/watch?v=jMropYKlcXU

8.1 Palabras aceptadas por un AFN

La palabra *w* es aceptada por un AFN si y solo si hay un camino en su gráfico de estado desde el estado inicial hasta el estado final tal que la concatenación de las etiquetas de flecha es igual a *w*.

Ejemplo. Vea si *baabbaba* se acepta por el AFN en la Figura 23.

La palabra *baabbaba* se puede dividir en cuatro partes: *p1 = b*, *p2 = a*, *p3 = abbab* y *p4 = a*, que juntas forman la palabra original. Ahora podemos monitorear la siguiente secuencia de estado (ruta) en el AFN, que se muestra en la Tabla 3.

Tabla 3

Secuencia de estados del AFN

Estado	Cadena que consume	Produce estado
1	b	1
1	a	1
1	abbab	2
2	a	2

Fuente: *Nota.* Elaboración propia.

De esta manera demostramos que la máquina *acepta* efectivamente la cadena *baabbaba*. Probar que una cadena no es aceptada por un AFN es más difícil porque se debe demostrar que ningún camino satisface los requisitos; el número de caminos posibles puede ser demasiado grande para ser considerado individualmente. En este ejemplo en particular, se puede ver que *ababab* tiene una cadena que no es aceptada por el autómata porque la transición asocia el estado inicial 1 con el estado final 2 que contiene dos *b* consecutivas que no están en la palabra, por lo que el estado final no puede ser alcanzada y la palabra no puede ser aceptada.

8.2 Diseño de los AFN

Un AFN puede usar técnicas de diseño modular para manejar mejor la complejidad del problema. Esos son los métodos modulares que describiremos más adelante.

8.2.1 AFN para la unión de lenguajes

Si tenemos dos AFN, *M1 y M2*, se pueden combinar para formar un nuevo AFN que acepte una combinación de lenguajes aceptados por ambos autómatas.

Sean $M1 = (K1, \Sigma1, \Delta1, s1, F1)$ y $M2 = (K2, \Sigma2, \Delta2, s2, F2)$ dos autómatas que aceptan los lenguajes *L1* y *L2*. Entonces podemos construir un AFN *M3* tomando $L1 \cup L2$ de la siguiente manera.

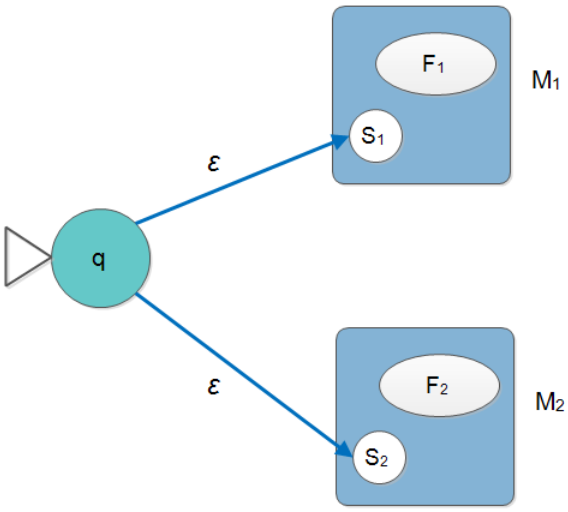
Sea *q* un nuevo estado que no existe en *K1* o *K2*. Luego creamos *M3* con el estado inicial *q* y transiciones vacías de *q* a *s1* y *s2*. Esta simple idea te permite elegir entre ir a *M1* o *M2*, lo que sea conveniente. Si la palabra de entrada *w* está en *L1*, elegimos reenviar a *M1* y de manera similar a *M2* para *L2*. En términos de formales:

$$M3 = (K1 \cup K2 \cup \{q\}, \Sigma1 \cup \Sigma2, \Delta1 \cup \Delta2 \cup \{(q, \varepsilon, s1), (q, \varepsilon, s2)\}, q, F1 \cup F2)$$

En la Figura 24 se representa gráficamente *M3*.

Figura 24

AFN para la unión de lenguajes

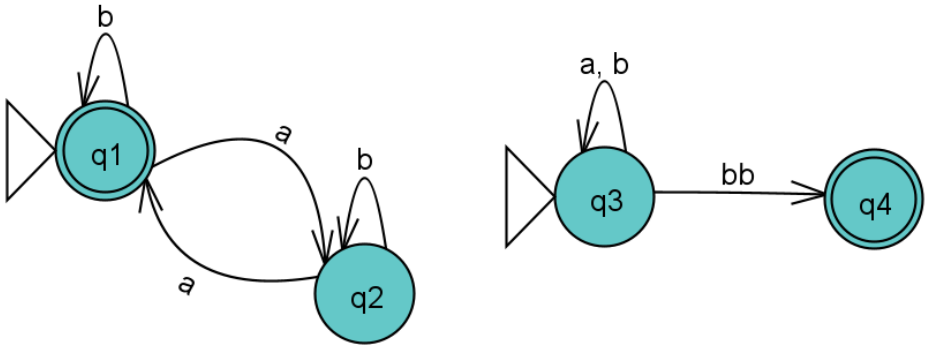


Fuente: *Nota.* Elaboración propia.

Ejemplo. Diseñaremos un AFN que acepte las palabras en {a, b} que contenga un numero par de *a's* o termine en *bb*. En la Figura 25(a) muestra que el AFN acepta palabras pares de *a's*, mientras que la Figura 25(b) muestra un segundo AFN que acepta palabras que terminan en *bb*.

Figura 25

AFN's a unir



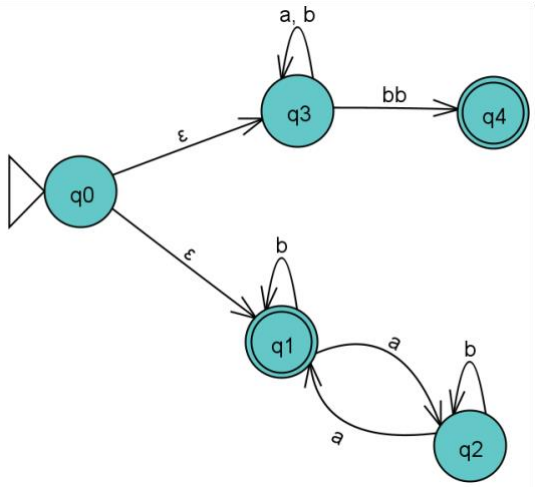
(a) Número par de *a's* (b) Terminan en *bb*

Fuente: *Nota.* Elaboración propia.

Finalmente, en la Figura 26 se muestra el AFN que acepta ambos lenguajes *L1y L2*.

Figura 26

Unión de AFN's



Fuente: *Nota.* Elaboración propia.

8.2.2 AFN para la concatenación de lenguajes

Como en el caso anterior, establezca *M1 = (K1, Σ1, ∆1, s1, F1)* y *M2 = (K2, Σ2, ∆2, s2, F2)* para que sean los dos lenguajes L1 y L2, respectivamente.

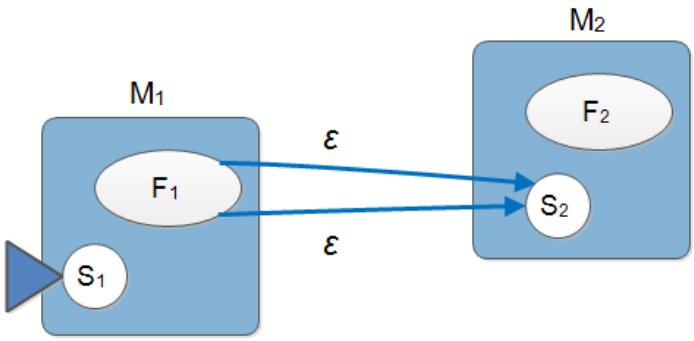
Entonces podemos crear un AFN *M3* que acepte L1L2. Agregamos una serie de transiciones vacías que conducen desde cada estado final de *M1* al estado inicial de M2; también se requiere que los estados finales de *M1* ya no sean estados finales. En términos de notación formal:

$M3 = (K1 \cup K2, \Sigma1 \cup \Sigma2, \Delta1 \cup \Delta2 \cup \{(p, \varepsilon, s2) / p \in F1\}, s1, F2)$

El trabajo de *M3* es el siguiente: cuando recibe la palabra *w = w1w2*, *w1* $\in$ *L1* y *w2* $\in$ *L2*, comienza a procesar *w1* exactamente de la misma manera que *M1* hasta que se alcanza uno de los antiguos estados finales de *M1*. entonces empezamos por tratar *w2* de la misma forma que *M2*; debe poder alcanzar el estado final *M2*, ya que para la condición M2 asume *w2*. La Figura 27 muestra el AFN *M3*.

Figura 27

AFN por medio de la concatenación de lenguajes



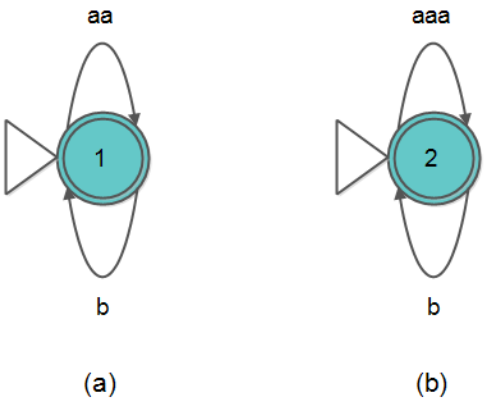
Fuente: *Nota.* Elaboración propia.

Ejemplo. Diseñamos un AFN que use el idioma en {a, b}, donde *a* está en grupos de al menos dos consecutivas, y a's en repeticiones de *aaa*, a la derecha de los lenguajes están la parte repetitiva de *aa*. De esta forma: *baabaaa*, *aaaaa*, *baab* o *baaaaa*. Y las condiciones que no se cumplen, por ejemplo, en *bbaaabaa* o *aaabaaaa*.

El AFN que se muestra en la Figura 28(a) acepta palabras que contienen los grupos b y aa en cualquier orden. El otro AFN en la Figura 28(b) usa un lenguaje similar, pero con grupos aaa.

Figura 28

AFN's a concatenar

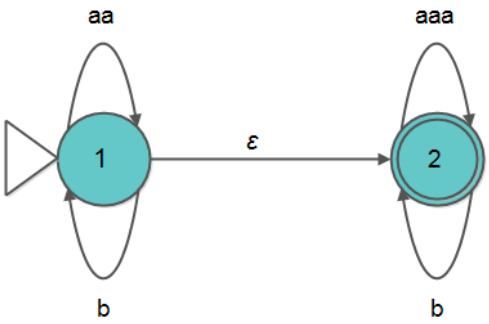


Fuente: *Nota.* Elaboración propia.

La solución es su concatenación, que se presenta en la Figura 29.

Figura 29

Concatenación de AFN's



Fuente: *Nota.* Elaboración propia.

Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender como se construye un autómata finito no deterministas (AFN):

Edgar Patricio Sánchez Malla. (13 de diciembre de 2020). *Autómatas finitos no deterministas - Ejercicio.* [Archivo de Vídeo]. Youtube. https://www.youtube.com/watch?v=FC161VU0iSk

Videos de elaboración propia



Referencias citadas en UNIDAD 2

- Hernández Rodríguez, L. A., Jaramillo Valbuena, S., & Cardona Torres, S. A. (2010). *Practique la teoría de autómatas y lenguajes formales*. Ediciones Elizcom.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2008). Introducción a la teoría de autómatas, lenguajes y computación. *Pearson Educación S.A.*
- Jurado, Elena. (2008). *Teorías de Autómatas y Lenguajes Formales*. Universidad de Extremadura.



Glosario de los términos citados en la UNIDAD 2

Autómata	Es un dispositivo que procesa cadenas de símbolos que recibe como entrada, cambiando de estado y produciendo una salida que, en algunos casos, puede estar formada por otra cadena de símbolos.
Autómata finito	Conjunto de estados y su control pasa de un estado a otro en respuesta a las entradas externas.
Autómata finito determinista (AFD)	Es aquel que sólo puede estar en un único estado después de leer cualquier secuencia de entradas. El término determinista hace referencia al hecho de que para cada entrada sólo existe uno y sólo un estado al que el autómata puede hacer la transición a partir de su estado actual.
Tabla de transiciones	Es una representación tabular convencional de una función.
Autómatas equivalentes	Dos autómatas M1 y M2 son equivalentes, M1 ≈ M2, cuando aceptan exactamente el mismo lenguaje.
Estados equivalentes	Dos estados son equivalentes, q1 ≈ q2, sí y sólo si al intercambiar uno por otro en cualquier configuración no altera la aceptación o rechazo de toda palabra.
Autómata finito no determinista (AFN)	Tiene la capacidad de estar en varios estados a la vez. Esta capacidad a menudo se expresa como la posibilidad de que el autómata conjeture algo acerca de su entrada